TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

BUỔI 9 · LÝ THUYẾT

Giao tiếp giữa các thành phần và Trình bày kết quả cho người dùng

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

NỘI DUNG

Năm phần

1 **Hợp đồng dữ liệu giữa các thành phần**

Hợp đồng là gì; ba thứ mọi hợp đồng phải nêu; phụ thuộc ngầm; kiểm tra dữ liệu ở ranh giới; mã trạng thái và định dạng lỗi của API; xử lý khi hợp đồng thay đổi.

2 **Trình bày kết quả cho người dùng cuối**

Người dùng cần gì ngoài con số; bốn nguyên tắc trình bày; đối sánh hai cách trình bày cùng một kết quả; ba trạng thái; thông báo lỗi cho người dùng.

3 **Tài liệu hoá trong mã: comment viết cho ai**

Ba loại comment đáng viết và một loại nên bỏ; tài liệu hoá hàm công khai (JSDoc, docstring); giữ comment không lạc hậu.

4 **Tích hợp module**

Tích hợp tăng dần thay vì ghép một lần; stub cho phần chưa xong; test hợp đồng ở ranh giới; phân loại lỗi tích hợp.

5 **Phần chuyên ngành – Sinh viên tự học theo tài liệu**

Frontend gọi API và quản lý trạng thái [SE] · nạp kho dữ liệu và dựng dashboard [DA] · huấn luyện, đọc metric và so sánh mô hình [AI]. Mini-case và tự kiểm tra cuối buổi.

MỤC TIÊU

Mục tiêu buổi học

1 Viết ra hợp đồng dữ liệu

- Nêu được hình dạng, ý nghĩa và hành vi khi sai ở mỗi ranh giới trong module của mình
- Viết hàm chuyển đổi ở ranh giới và kiểm tra dữ liệu đầu vào tại đó

2 Trình bày kết quả có ngữ cảnh

- Áp dụng bốn nguyên tắc trình bày cho một màn hình, dashboard hoặc báo cáo mô hình
- Xử lý đủ ba trạng thái: đang tải, rỗng, lỗi

3 Tài liệu hoá đúng chỗ

- Viết comment giải thích VÌ SAO và ghi mã quy tắc nghiệp vụ
- Viết JSDoc hoặc docstring cho hàm công khai

4 Tích hợp tăng dần

- Ghép các module theo luồng chính, từng mối nối một
- Viết ít nhất một test hợp đồng ở ranh giới

Bốn việc này là đầu vào trực tiếp cho thành phần 3 của Bài tập 2 (module mở rộng và phần trình bày kết quả) — làm ở workshop buổi 10, nộp cuối buổi 12.

01 Hợp đồng dữ liệu giữa các thành phần

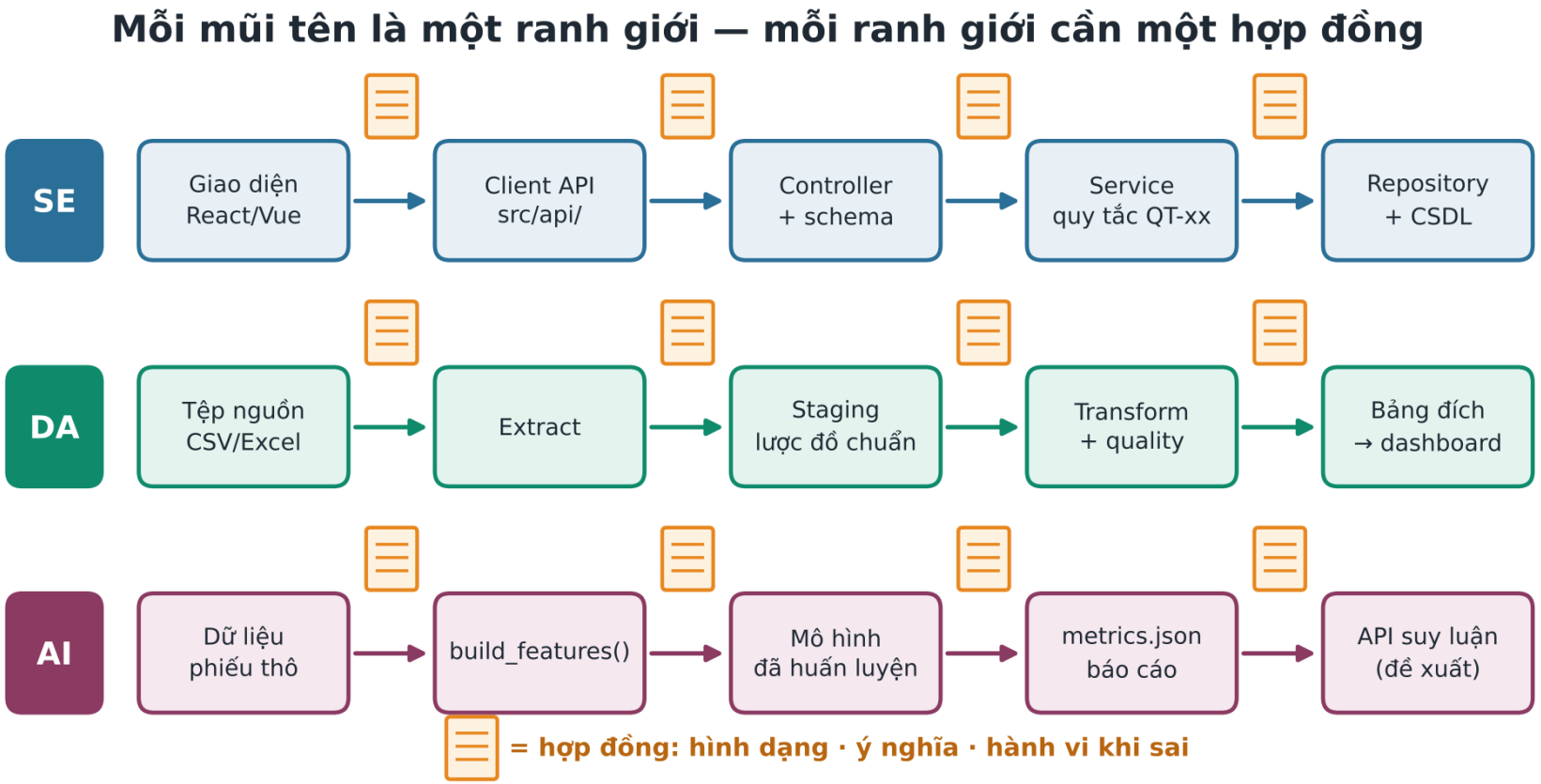
Hợp đồng là gì · phụ thuộc ngầm · chuyển đổi ở ranh giới · khi hợp đồng thay đổi

KHÁI NIỆM CỐT LÕI CỦA BUỔI HỌC

Hai thành phần chỉ ghép được với nhau khi cả hai đồng ý về hình dạng của dữ liệu chảy qua ranh giới giữa chúng.

- HỢP ĐỒNG DỮ LIỆU (data contract) là thoả thuận về: dữ liệu gồm những trường nào, mỗi trường kiểu gì, trường nào bắt buộc, và ý nghĩa của từng giá trị.
- Hợp đồng tồn tại ở mọi ranh giới, không chỉ ở API: giữa lớp trình bày và lớp nghiệp vụ, giữa lớp nghiệp vụ và lớp dữ liệu, giữa bước extract và bước transform của một pipeline, giữa bước tạo đặc trưng và bước huấn luyện mô hình.
- Vấn đề là: hợp đồng LUÔN tồn tại, nhưng nó có thể không được viết ra. Khi đó nó trở thành PHỤ THUỘC NGẦM — và mọi thay đổi đều có nguy cơ làm vỡ một chỗ khác mà không ai biết.
- Công việc của kỹ sư ở buổi này là biến các hợp đồng ngầm thành hợp đồng tường minh.

Chuỗi thành phần và các ranh giới



Đọc hình thế nào

- Mỗi mũi tên là một chỗ dữ liệu đi qua — tức là một hợp đồng, dù đã viết ra hay chưa.
- SE: hợp đồng giữa client API và controller là định dạng JSON và mã lỗi.
- DA: hợp đồng giữa extract và transform là lược đồ staging.
- AI: build_features() là ranh giới nguy hiểm nhất — lệch thứ tự cột thì mô hình sai im lặng.
- Bài tập: tìm MỘT mũi tên trong module của mình và viết hợp đồng cho nó.

BA THỨ MỌI HỢP ĐỒNG PHẢI NÊU RÕ

Ba thứ phải viết ra ở mỗi ranh giới

1

HÌNH DẠNG — có những trường nào, kiểu gì, trường nào bắt buộc

Không chỉ tên trường mà cả kiểu và tính bắt buộc. "ngayTiepNhan: chuỗi ISO 8601 kèm múi giờ, bắt buộc" khác hẳn "ngayTiepNhan: ngày, có thể để trống".

2

Ý NGHĨA — mỗi giá trị nghĩa là gì, đơn vị là gì

thoiGianXuLy = 72 nghĩa là 72 giờ hay 72 phút? conBaoHanh = null nghĩa là hết bảo hành hay chưa xác minh được? Đây là nơi lỗi khó tìm nhất sinh ra.

3

HÀNH VI KHI SAI — không có dữ liệu thì trả về gì, lỗi thì báo thế nào

Không tìm thấy khách hàng thì trả về null, trả về mảng rỗng, hay ném lỗi? Cả hai phía phải biết trước. Đây là chỗ sinh ra lỗi "gọi thuộc tính của giá trị rỗng" kinh điển.

Cùng một lỗi, ba track đều gặp

TRƯỚC: hợp đồng ngầm → SAU: hợp đồng tường minh

```
// ===== TRƯỚC — hợp đồng ngầm, dễ vỡ =====
// Lớp trình bày tự đoán cấu trúc dữ liệu mà service trả về.
const ds = await ticketService.layDanhSach();
ds.forEach(t => hienThi(t.customer.name, t.dueDate));
// → Nếu service đổi t.customer thành t.khachHang, chỗ này vỡ LÚC CHẠY,
//   không ai biết cho tới khi người dùng mở trang.

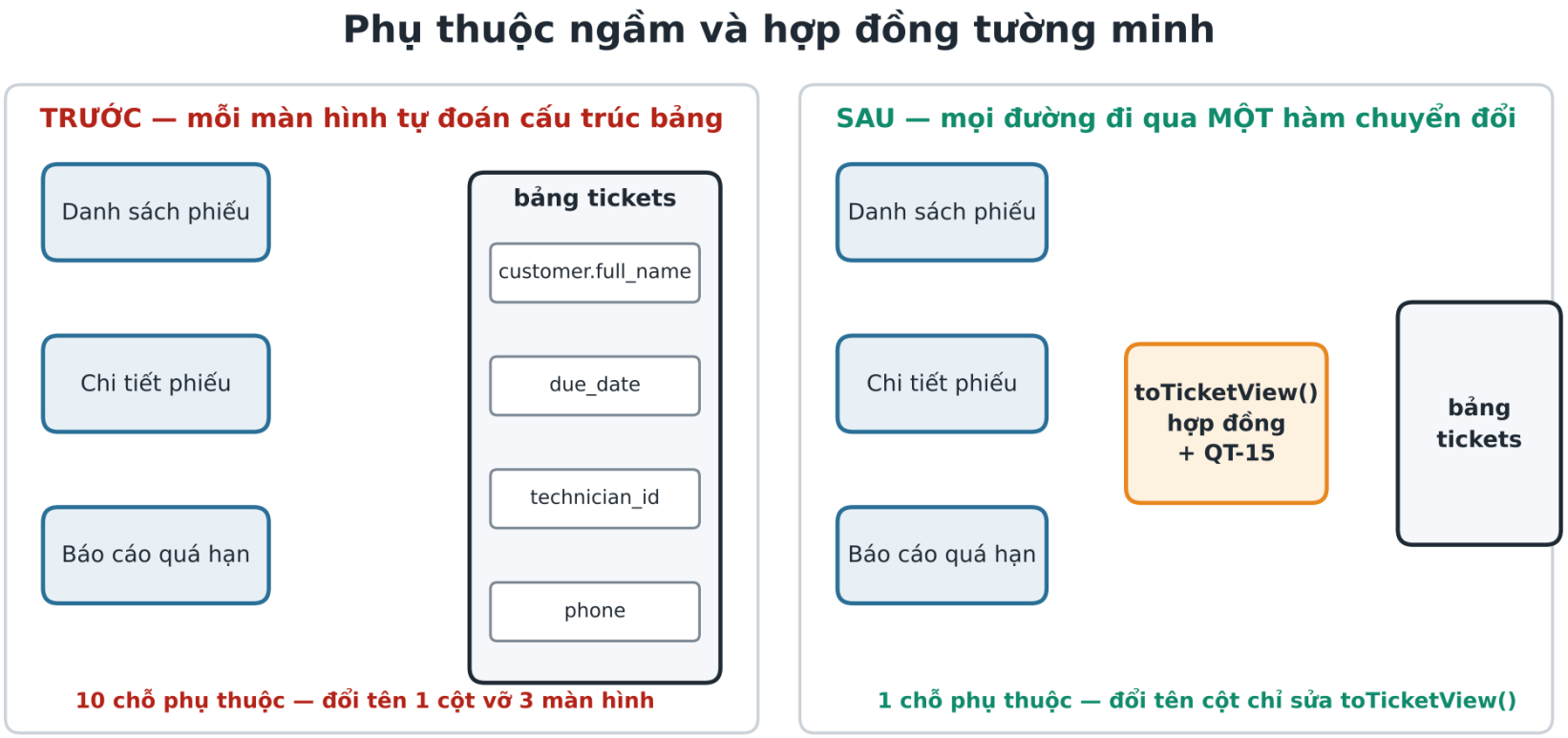
// ===== SAU — hợp đồng tường minh =====
// src/service/ticket.contract.js
// Hợp đồng: dữ liệu MỘT PHIẾU trả ra từ lớp nghiệp vụ.
//   ticketId   : số nguyên, luôn có
//   ticketCode : chuỗi dạng BH-000123/2026, luôn có
//   tenKhach   : chuỗi, luôn có (đã che số điện thoại theo QT-15)
//   hanCamKet  : chuỗi ISO 8601 kèm múi giờ, luôn có
//   kyThuatVien: chuỗi HOẶC null nếu chưa phân công
function toTicketView(t) {
  return {
    ticketId: t.id,
    ticketCode: t.ticket_code,
    tenKhach: t.customer?.full_name ?? '(không rõ)',
    hanCamKet: t.due_date.toISOString(),
    kyThuatVien: t.technician?.full_name ?? null,
  };
}
```

Ghi nhớ

- Hàm chuyển đổi ở ranh giới (toTicketView) là nơi hợp đồng được viết ra và được thi hành.
- Lớp trình bày KHÔNG bao giờ chạm vào cấu trúc bảng cơ sở dữ liệu — nó chỉ biết hợp đồng.
- Đổi tên cột trong cơ sở dữ liệu chỉ phải sửa MỘT chỗ.
- [DA] Tương đương: một hàm chuẩn hoá lược đồ DataFrame giữa extract và transform.
- [AI] Tương đương: một hàm build_features() trả về đúng danh sách cột theo đúng thứ tự.
- Đây cũng là nơi áp dụng QT-15 (che số điện thoại) — một chỗ duy nhất.

MINH HOẠ — PHỤ THUỘC NGẦM

Mười đường phụ thuộc, hay một?



Đọc hình thế nào

- Bên trái: mỗi màn hình đọc thẳng tên cột của bảng — 10 đường phụ thuộc mà không ai ghi lại.
- Đổi tên một cột (ví dụ due_date) làm vỡ cùng lúc nhiều màn hình, và chỉ lộ ra LÚC CHẠY.
- Bên phải: mọi đường đi qua toTicketView() — hợp đồng được viết và thi hành ở MỘT chỗ.
- Cũng là chỗ duy nhất áp dụng QT-15 (che số điện thoại), nên không thể bị sót.

HỢP ĐỒNG Ở MỌI TRACK

Ranh giới nào, hợp đồng nằm ở đâu, thi hành bằng gì

Track	Ranh giới điển hình	Hợp đồng viết ở đâu	Thi hành bằng	Nếu bỏ qua
SE	Frontend ↔ API Controller ↔ Service ↔ Repository	Tệp *.contract.js / schema; tài liệu API (README hoặc OpenAPI)	Hàm chuyển đổi toXxxView; kiểm tra đầu vào bằng zod hoặc Joi	Giao diện vỡ lúc chạy khi backend đổi tên trường
DA	Tệp nguồn ↔ extract extract ↔ transform ↔ load	Data dictionary; lược đồ staging; tệp schema.yaml	Hàm chuẩn hoá lược đồ; kiểm tra cột, kiểu, khoá bằng pandera hoặc assert	Dashboard hiện số sai mà không ai phát hiện
AI	Dữ liệu thô ↔ build_features build_features ↔ mô hình ↔ API suy luận	Danh sách cột đặc trưng theo thứ tự; nhãn hợp lệ; lưu kèm mô hình	build_features() duy nhất cho cả huấn luyện và suy luận; kiểm tra đầu vào bằng pydantic	Mô hình dự đoán sai mà KHÔNG báo lỗi

Câu hỏi tự kiểm: “Nếu ngày mai một bên đổi tên một trường, có bao nhiêu chỗ phải sửa?” — câu trả lời đúng là MỘT: hàm chuyển đổi ở ranh giới.

Kiểm tra một lần ở cửa vào, tin tưởng ở bên trong

Cùng một hợp đồng, viết bằng zod (JavaScript) và pydantic (Python)

```
// ===== SE — src/validators/ticket.schema.js (zod) =====
import { z } from 'zod';
export const TiepNhanSchema = z.object({
  serialNo:  z.string().regex(/^[A-Z0-9]{8,15}$/, 'Số serial không hợp lệ'),
  mucDo:     z.enum(['CAO', 'TRUNG_BINH', 'THAP']),   // QT-04
  moTaLoi:   z.string().min(10, 'Mô tả lỗi tối thiểu 10 ký tự'),
  ngayMua:   z.string().date().nullable(),            // null = chưa xác minh
});
// Controller: kiểm tra Ở ĐÂY, service nhận dữ liệu đã sạch.
const kq = TiepNhanSchema.safeParse(req.body);
if (!kq.success) return res.status(422).json(loiHopLe(kq.error));

# ===== AI — src/api/schemas.py (pydantic) =====
from pydantic import BaseModel, Field
class YeuCauPhanLoai(BaseModel):
    mo_ta_loi: str = Field(min_length=10, max_length=2000)
    dong_may:  str | None = None        # None = khách không nhớ dòng máy
# FastAPI tự trả 422 kèm danh sách trường sai nếu đầu vào không khớp.
```

Ghi nhớ

- Kiểm tra ở RANH GIỚI (controller, đầu pipeline, đầu API suy luận) — không rải if khắp lớp nghiệp vụ.
- Schema chính là hợp đồng được viết ra bằng mã: vừa là tài liệu, vừa được thi hành.
- Thông báo lỗi trong schema viết cho người dùng, bằng tiếng Việt có dấu.
- Trả 422 kèm danh sách trường sai, không trả 500.
- [DA] Tương đương: pandera.DataFrameSchema kiểm tra cột, kiểu, khoá trước bước transform.

Một định dạng lỗi cho toàn bộ API

Mã HTTP	Khi nào dùng	Thân phản hồi mẫu	Bên gọi phải làm gì
200 / 201	Thành công; 201 khi vừa tạo mới tài nguyên	{ "data": { "ticketCode": "BH-000123/2026", ... } }	Hiển thị dữ liệu; với 201 chuyển tới trang chi tiết
400 / 422	Đầu vào sai hình dạng (400) hoặc sai kiểu, thiếu trường (422)	{ "error": "DU_LIEU_KHONG_HOP_LE", "message": "...", "fields": { "moTaLoi": "Tối thiểu 10 ký tự" } }	Hiện lỗi ngay cạnh từng ô nhập; KHÔNG thử lại
401 / 403	Chưa đăng nhập hoặc token hết hạn (401); không đủ quyền (403)	{ "error": "CHUA_DANG_NHAP", "message": "Phiên làm việc đã hết hạn" }	401: xoá token, về trang đăng nhập · 403: báo không đủ quyền
404 / 409	Không tìm thấy (404); xung đột quy tắc nghiệp vụ (409)	{ "error": "PHIEU_DANG_MO", "message": "Thiết bị này đang có phiếu chưa hoàn tất (QT-03)" }	Hiện thông điệp nghiệp vụ và lối đi tiếp (mở phiếu đang có)
500 / 503	Lỗi hệ thống không lường trước; dịch vụ tạm thời không sẵn sàng	{ "error": "LOI_HE_THONG", "message": "Hệ thống đang gặp sự cố. Vui lòng thử lại." }	Hiện thông báo chung kèm nút thử lại; KHÔNG hiện chi tiết kỹ thuật

Ba trường cố định — error (mã máy đọc), message (câu người đọc), fields (lỗi theo trường, nếu có). Frontend dựa vào error để rẽ nhánh, không dựa vào câu chữ của message.

KHI HỢP ĐỒNG THAY ĐỔI

Đổi hợp đồng mà không làm vỡ bên kia

1

Phân loại thay đổi: tương thích hay phá vỡ

Tương thích: thêm trường TUỲ CHỌN, thêm giá trị mới mà bên gọi bỏ qua được. Phá vỡ: đổi tên trường, đổi kiểu, đổi đơn vị, xoá trường, biến trường tuỳ chọn thành bắt buộc, đổi ý nghĩa của null.

2

Thay đổi phá vỡ thì làm theo ba bước: mở rộng → chuyển → thu hẹp

Thêm trường mới song song với trường cũ; chuyển từng bên gọi sang trường mới; chỉ xoá trường cũ khi không còn ai dùng. Không bao giờ đổi tên tại chỗ trong một commit.

3

Tìm mọi bên đang dùng hợp đồng trước khi sửa

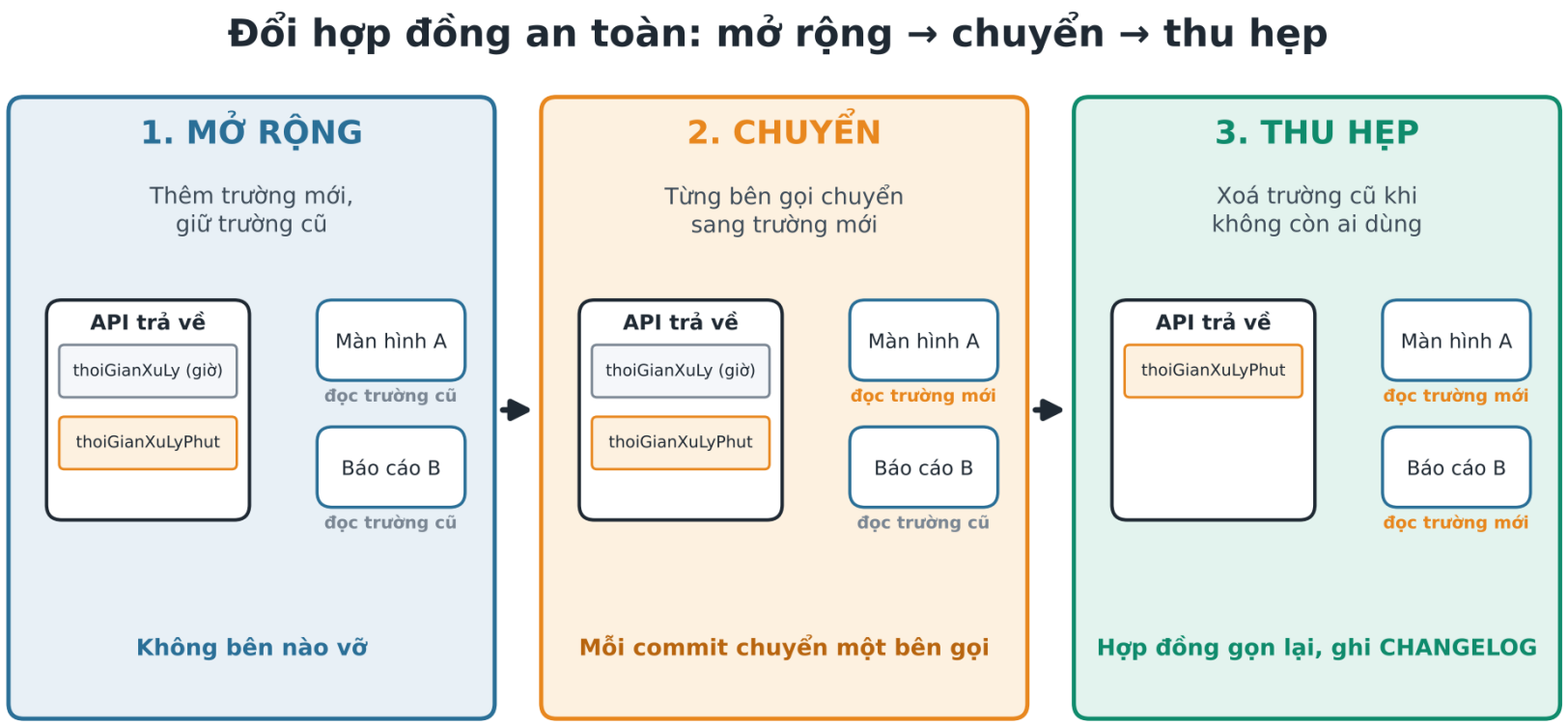
Tìm kiếm toàn repo theo tên trường; chạy toàn bộ test; với DA kiểm tra mọi truy vấn dashboard; với AI kiểm tra mô hình đã lưu có còn khớp danh sách cột không.

4

Ghi lại thay đổi ở nơi người khác sẽ đọc

Cập nhật comment hợp đồng, schema và tài liệu API trong CÙNG commit với mã; commit message nêu rõ “đổi hợp đồng”; nếu là thay đổi thiết kế so với BT1 thì ghi vào mục điều chỉnh thiết kế.

Ba bước đổi hợp đồng mà không bên nào vỡ



Sai lầm: đổi tên hoặc đổi đơn vị TẠI CHỖ trong một commit — mọi bên gọi cũ vỡ cùng lúc

Đọc hình thế nào

- Ví dụ: đổi thoiGianXuLy từ GIỜ sang PHÚT — tên không đổi nhưng ý nghĩa đổi, nên đây là thay đổi phá vỡ.
- Bước 1 thêm trường mới; bước 2 chuyển từng bên gọi, mỗi commit một bên; bước 3 mới xoá trường cũ.
- Ở mọi thời điểm, cả hai bên đều chạy được — không có "ngày vỡ".
- [AI] Tương tự khi đổi build_features: lưu mô hình mới song song, chuyển API suy luận, rồi mới bỏ mô hình cũ.

02 Trình bày kết quả cho người dùng cuối

Người dùng cần gì ngoài con số · bốn nguyên tắc · ba trạng thái mọi màn hình phải có

NGƯỜI DÙNG CẦN GÌ NGOÀI CON SỐ

Mỗi kết quả phải trả lời một câu hỏi của một người cụ thể

Track	Người dùng cuối	Câu hỏi họ mang tới	Kết quả phải cho họ
SE	Nhân viên tiếp nhận, kỹ thuật viên, quản lý trung tâm	“Hôm nay tôi phải xử lý phiếu nào trước?”	Danh sách phiếu sắp theo hạn cam kết, đánh dấu phiếu sắp quá hạn, có nút thao tác ngay trên dòng
DA	Quản lý vận hành, ban giám đốc	“Trung tâm nào đang có vấn đề, và vấn đề lớn tới đâu?”	Biểu đồ so sánh có mốc tham chiếu (trung bình, mục tiêu SLA), nguồn và thời điểm dữ liệu
AI	Nhân viên tiếp nhận dùng gợi ý phân loại; người duy trì mô hình	“Gợi ý này có đáng tin không? Khi nào tôi nên tự quyết?”	Nhãn đề xuất kèm độ tin cậy, ghi rõ là ĐỀ XUẤT; báo cáo nêu nhãn nào mô hình hay sai

Ba thứ người dùng luôn cần ngoài con số: NGỮ CẢNH (so với cái gì) · ĐỘ TIN CẬY (dữ liệu nào, tới lúc nào, chắc tới đâu) · HÀNH ĐỘNG (tiếp theo làm gì).

Bốn nguyên tắc, áp dụng cho mọi track

1

Trả lời câu hỏi của người dùng, không phô dữ liệu

Quản lý hỏi "phiếu nào sắp quá hạn". Đừng đưa bảng 500 phiếu rồi để họ tự lọc.

Đưa đúng danh sách phiếu sắp quá hạn, sắp theo mức khẩn.

2

Con số phải có ngữ cảnh để so sánh

"CSAT 3,86" không nói lên gì.

"CSAT 3,86 — giảm 0,2 so với tháng trước, thấp hơn mức mục tiêu 4,0" thì hành động được.

3

Nêu rõ giới hạn và độ tin cậy của kết quả

Kết quả tính trên dữ liệu nào, tới thời điểm nào, bỏ sót gì.

[AI] Đề xuất của mô hình phải kèm độ tin cậy và ghi rõ đây là ĐỀ XUẤT.

4

Dẫn tới hành động tiếp theo

Sau khi thấy kết quả, người dùng làm gì?

Màn hình phiếu quá hạn nên có nút phân công lại ngay tại đó.

ĐỐI SÁNH HAI CÁCH TRÌNH BÀY CÙNG MỘT KẾT QUẢ

Cùng một truy vấn, hai giá trị sử dụng rất khác nhau

Bản A — đổ dữ liệu ra màn hình

```
center_id | avg_hrs   | late_cnt
-------------------------------
  CT01    | 61.2391   | 14
  HCM03   | 88.7004   | 37
  DN02    | 52.0118   | 9
  ...     (21 dòng nữa)
```

Tên cột của cơ sở dữ liệu, số lẻ 4 chữ số, không sắp xếp, không có mốc so sánh, không biết số liệu tính tới bao giờ, không có việc gì để làm tiếp.

Bản B — trả lời câu hỏi của quản lý

- **Tiêu đề là câu kết luận:** "3 trung tâm xử lý chậm hơn mục tiêu SLA (***Service Level Agreement***) 72 giờ"
- **Sắp theo mức độ nghiêm trọng:** HCM03 88,7 giờ (vượt 23%) đứng đầu
- **Có mốc so sánh:** đường trung bình hệ thống 64,5 giờ và đường mục tiêu 72 giờ
- **Nêu giới hạn:** "Tính trên 1.284 phiếu đã đóng, 01/01–30/06/2026; chưa gồm phiếu đang mở"
- **Dẫn tới hành động:** bấm vào HCM03 để xem 37 phiếu trễ hạn, phân công lại kỹ thuật viên

Bản B không cần truy vấn mới — chỉ cần làm tròn, đặt tên cho người đọc, sắp xếp, thêm mốc so sánh, ghi nguồn và thêm một lối đi tiếp.

Bản B trông như thế nào trên một dashboard thật

Cùng một truy vấn — Bản A và Bản B

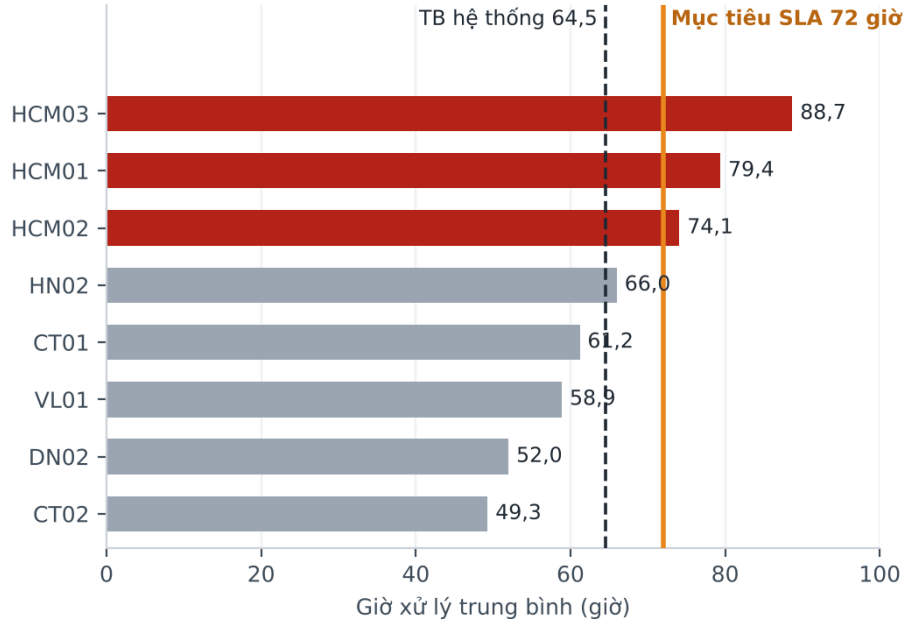
Bản A — đổ dữ liệu

center_id	avg_hrs	late
CT01	61.2391	14
HCM03	88.7004	37
DN02	52.0118	9
HCM01	79.4410	25
HN02	66.0193	18
VL01	58.8820	11
HCM02	74.1260	22
CT02	49.3307	6

Không sắp xếp · không mốc · không nguồn

→

Bản B: 3 trung tâm chậm hơn mục tiêu SLA 72 giờ



Nguồn: fact_ticket · 1.284 phiếu đã đóng, 01/01–30/06/2026 · bấm vào trung tâm để xem phiếu

Đọc hình thế nào

- Tiêu đề biểu đồ là CÂU KẾT LUẬN, không phải tên bảng.
- Sắp xếp theo giá trị; chỉ tô đỏ những trung tâm vượt mục tiêu — mắt đi thẳng tới vấn đề.
- Hai mốc tham chiếu: trung bình hệ thống (đứt nét) và mục tiêu SLA (cam).
- Dòng nguồn cuối hình: dữ liệu nào, giai đoạn nào, bấm vào đâu để làm tiếp.
- Số làm tròn một chữ số, dấu phẩy thập phân kiểu Việt Nam.

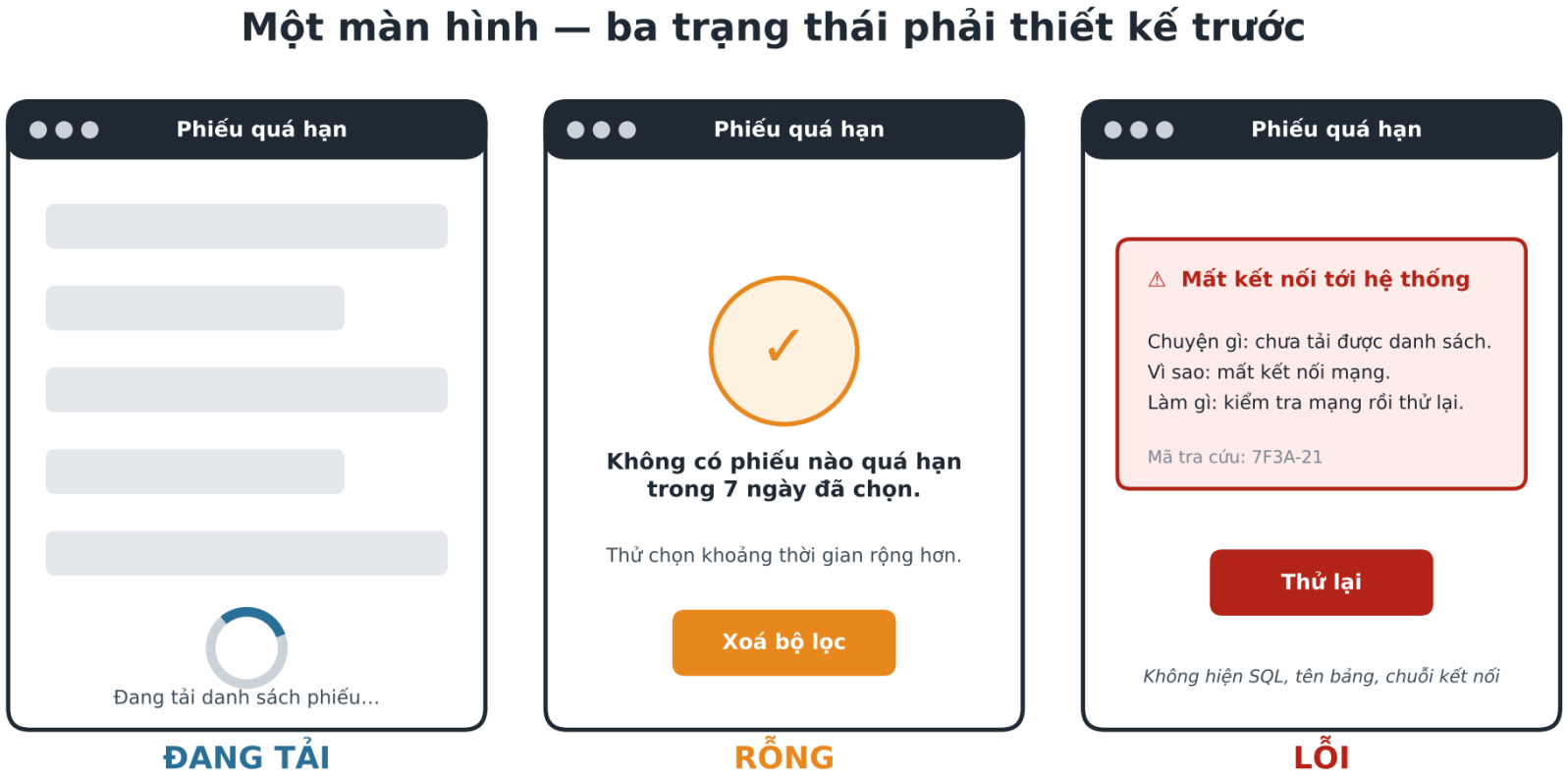
BA TRẠNG THÁI MÀ MỌI MÀN HÌNH VÀ MỌI BÁO CÁO PHẢI XỬ LÝ

Ba trạng thái sinh viên hay quên

Trạng thái	Xảy ra khi nào	Phải hiển thị gì	Lỗi thường gặp
ĐANG TẢI	Đang chờ dữ liệu từ máy chủ hoặc đang chạy truy vấn nặng	Dấu hiệu cho biết hệ thống đang làm việc; nếu lâu thì cho biết đang ở bước nào	Màn hình trắng — người dùng tưởng hệ thống hỏng và bấm lại nhiều lần
RỖNG	Truy vấn chạy đúng nhưng không có dữ liệu nào thoả điều kiện	Câu giải thích vì sao rỗng và gợi ý hành động: "Không có phiếu nào quá hạn trong khoảng thời gian đã chọn"	Bảng trống trơn — người dùng không phân biệt được "không có dữ liệu" với "hệ thống lỗi"
LỖI	Mất kết nối, hết thời gian chờ, hoặc lỗi hệ thống	Thông báo có ba phần: chuyện gì, vì sao, làm gì tiếp; kèm nút thử lại	Hiện nguyên thông báo lỗi kỹ thuật của cơ sở dữ liệu ra màn hình người dùng

Ba trạng thái này áp dụng cho cả ba track: một dashboard [DA] cũng phải xử lý trường hợp bộ lọc không ra dòng nào; một API suy luận [AI] cũng phải trả lời rõ ràng khi đầu vào không hợp lệ thay vì trả về lỗi 500.

Thiết kế cả ba khung hình, không chỉ khung “có dữ liệu”



Đọc hình thế nào

- ĐANG TẢI: khung xương (skeleton) giữ bố cục, cho người dùng biết hệ thống đang làm việc.
- RỖNG: nói rõ vì sao rỗng và đưa một hành động (xoá bộ lọc, đổi khoảng thời gian).
- LỖI: chuyện gì · vì sao · làm gì tiếp, kèm nút thử lại và mã tra cứu.
- [DA] Dashboard cũng cần cả ba khung; [AI] API suy luận trả 422 có thông điệp, không trả 500.

THÔNG BÁO LỖI CHO NGƯỜI DÙNG

Chuyện gì · vì sao · làm gì tiếp

Tình huống	Thông báo tệ	Thông báo tốt
Mất kết nối tới máy chủ	TypeError: Failed to fetch	Không kết nối được tới hệ thống. Kiểm tra mạng rồi bấm Thử lại. Dữ liệu đã nhập vẫn được giữ.
Trùng phiếu theo QT-03	Error 409	Thiết bị SN12345678 đang có phiếu BH-000098/2026 chưa hoàn tất. Mở phiếu đó để cập nhật thay vì tạo phiếu mới.
Bộ lọc dashboard không ra dòng nào [DA]	(biểu đồ trống, không chữ)	Không có phiếu nào của Trung tâm Cần Thơ trong tháng 7/2026. Thử chọn khoảng thời gian rộng hơn.
Đầu vào quá ngắn cho mô hình [AI]	500 Internal Server Error	Mô tả lỗi cần ít nhất 10 ký tự để hệ thống gợi ý nhóm lỗi. Nhân viên vẫn có thể chọn nhóm lỗi thủ công.
Lỗi không lường trước	duplicate key value violates unique constraint "tickets_pkey"	Hệ thống đang gặp sự cố khi lưu phiếu. Vui lòng thử lại sau ít phút. Mã tra cứu: 7F3A-21 (ghi log, không hiện chi tiết kỹ thuật).

Không bao giờ hiện thông báo kỹ thuật, tên bảng, câu SQL hay chuỗi kết nối ra màn hình người dùng — ghi chúng vào log kèm một mã tra cứu để đối chiếu.

03 Tài liệu hoá inline: comment viết cho ai

Comment tốt và comment thừa · ba loại comment đáng viết · giữ comment không lạc hậu

Ba loại comment đáng viết, một loại nên bỏ

ĐÁNG VIẾT 1 — Giải thích VÌ SAO, không giải thích CÁI GÌ.

- Mã đã nói được cái gì. Comment nên nói lý do: "// Dùng khoá nhân tạo thay vì số điện thoại vì khách có thể đổi số — xem QT-01".

ĐÁNG VIẾT 2 — Ghi mã quy tắc nghiệp vụ để giữ truy vết.

- "// QT-04: hạn cam kết CAO = 24h, TRUNG_BINH = 72h, THAP = 120h". Người đọc tra ngược được về bản SRS.

ĐÁNG VIẾT 3 — Cảnh báo cái bẫy hoặc quyết định đánh đổi.

- "// CẢNH BÁO: hàm này giả định danh sách đã sắp theo hạn cam kết. Gọi sau khi sắp xếp." hoặc "// Chấp nhận truy vấn N+1 ở đây vì danh sách tối đa 20 dòng; tối ưu khi vượt ngưỡng đó."

NÊN BỎ — Comment chỉ lặp lại mã bằng tiếng Việt.

- "// tăng i lên 1" trên dòng i++. Loại comment này không thêm thông tin và chắc chắn sẽ lạc hậu khi mã đổi.

Comment lạc hậu tệ hơn không có comment

- Một comment mô tả sai hành vi hiện tại của mã sẽ đánh lừa người đọc, kể cả chính người viết ba tháng sau.
- Quy tắc: sửa mã thì sửa comment ngay trong cùng một commit.
- Nếu comment khó viết vì logic quá rối, hãy sửa logic thay vì viết comment dài giải thích nó.
- Comment ghi mã quy tắc giúp Sinh viên nhớ lại nhanh.

Hàm công khai cần một hợp đồng viết ngay trên đầu

JSDoc (JavaScript) và docstring (Python) — cùng bốn phần

```
/**
 * Kiểm tra thiết bị còn trong thời hạn bảo hành hay không (QT-05).
 *
 * @param {{ngayMua: string|null, soThangBaoHanh: number}} thietBi
 * @param {string} ngayKiemTra  Ngày dạng YYYY-MM-DD, thường là ngày tiếp nhận.
 * @returns {{conBaoHanh: boolean|null, lyDo: string}}
 *   conBaoHanh = null khi CHƯA XÁC MINH được (thiếu ngày mua).
 * @throws {RangeError} Khi soThangBaoHanh âm — lỗi dữ liệu, không phải lỗi người dùng.
 */
export function kiemTraConBaoHanh(thietBi, ngayKiemTra) { ... }
```

```
def build_features(df: pd.DataFrame) -> pd.DataFrame:
    """Tạo ma trận đặc trưng cho mô hình phân loại nhóm lỗi.

    Dùng CHUNG cho huấn luyện và suy luận — không được viết bản thứ hai.

    Args:
        df: phải có cột mo_ta_loi (str) và dong_may (str, có thể rỗng).
    Returns:
        DataFrame với đúng các cột trong FEATURE_COLUMNS, đúng thứ tự đó.
    Raises:
        KeyError: nếu thiếu cột mo_ta_loi.
    """
```

Bốn phần bắt buộc

- **Làm gì** — một câu, kèm mã quy tắc nghiệp vụ nếu có.
- **Nhận gì** — kiểu và ràng buộc của từng tham số.
- **Trả gì** — kể cả ý nghĩa của null, mảng rỗng, giá trị đặc biệt.
- **Khi nào lỗi** — loại lỗi ném ra và vì sao.
- Chỉ viết cho hàm CÔNG KHAI (được module khác gọi). Hàm nội bộ ngắn thì tên hàm tốt là đủ.
- Trình soạn thảo hiện JSDoc/docstring khi rê chuột — người gọi không cần mở mã nguồn.

04 Tích hợp module

Ghép tăng dần · stub cho phần chưa xong · test hợp đồng ở ranh giới · phân loại lỗi tích hợp

TÍCH HỢP TĂNG DẦN

Ghép từng mối nối, không ghép một lần

1

Vẽ luồng chính thành chuỗi mối nối

Lấy luồng chính trong đặc tả use case ở BT1, viết thành chuỗi: giao diện → API → service → repository → CSDL [SE]; tệp nguồn → staging → bảng đích → dashboard [DA]; dữ liệu → đặc trưng → mô hình → API suy luận [AI].

2

Dùng stub cho phần chưa xong — đừng chờ

Phần phía sau chưa có thì thay bằng một hàm trả dữ liệu mẫu ĐÚNG HỢP ĐỒNG. Ví dụ: frontend gọi một API giả trả đúng hình dạng phiếu. Khi phần thật xong, chỉ việc thay stub.

3

Ghép một mối nối, chạy thử, rồi mới ghép tiếp

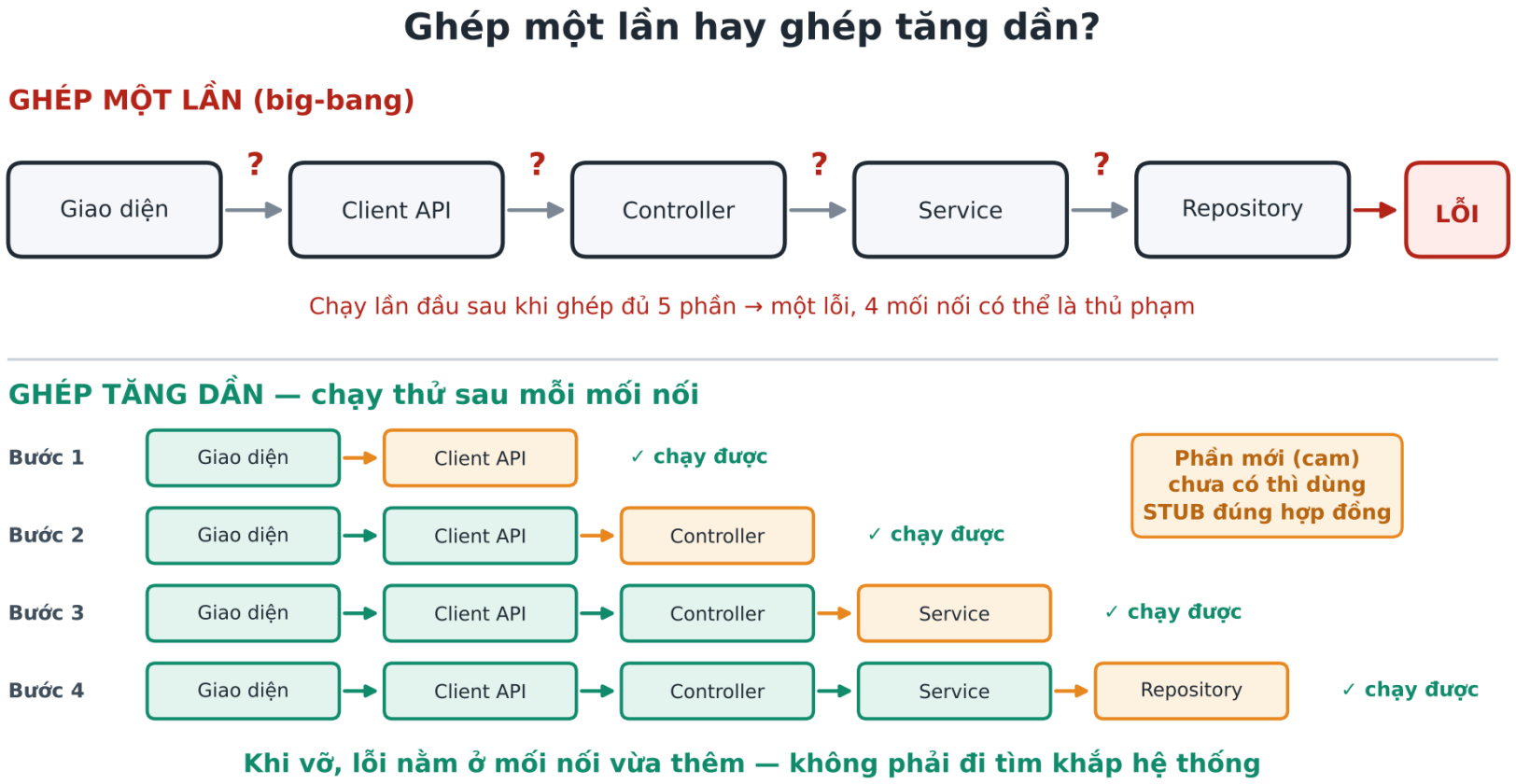
Mỗi lần chỉ thêm một mối nối. Nếu vỡ, lỗi nằm ở mối nối vừa thêm. Ghép tất cả rồi mới chạy (big-bang) cho một lỗi không biết đến từ đâu.

4

Chạy toàn luồng trên dữ liệu thật và lưu bằng chứng

Dùng dữ liệu từ bộ dataset Mekong Mobile, không phải ba dòng tự gõ. Ghi lại đầu vào, đầu ra và chụp màn hình kết quả vào docs/ — đây là minh chứng cho mục kiểm chứng BT2.

Ghép một lần, hay ghép từng mối nối?



Đọc hình thế nào

- Big-bang: chạy lần đầu khi đủ 5 phần — một lỗi, bốn mối nối có thể là thủ phạm.
- Tăng dần: mỗi bước chỉ thêm MỘT mối nối và chạy thử ngay.
- Phần mới (cam) chưa có thì dùng stub trả dữ liệu đúng hợp đồng.
- [DA] tệp nguồn → staging → bảng đích → dashboard; [AI] dữ liệu → đặc trưng → mô hình → metrics: cùng cách ghép.

Một test nhỏ giữ cho hai bên không lệch nhau

Kiểm tra HÌNH DẠNG đầu ra, không kiểm tra giao diện

```
// ===== SE — tests/ticket.contract.test.js (Jest) =====
import { toTicketView } from '../src/service/ticket.contract';
test('toTicketView trả đúng các trường của hợp đồng', () => {
  const v = toTicketView(phieuMauTuCSDL());          // dữ liệu mẫu từ seeds/
  expect(Object.keys(v).sort()).toEqual(
    ['hanCamKet', 'kyThuatVien', 'tenKhach', 'ticketCode', 'ticketId']);
  expect(v.hanCamKet).toMatch(/^\d{4}-\d{2}-\d{2}T/);   // ISO 8601
  expect(v.tenKhach).not.toMatch(/\d{7,}/);           // QT-15: đã che số
});
test('kyThuatVien là null khi chưa phân công', () => {
  expect(toTicketView({ ...phieuMauTuCSDL(), technician: null })
    .kyThuatVien).toBeNull();
});

# ===== AI — tests/test_features.py (pytest) =====
def test_build_features_dung_cot_dung_thu_tu(df_mau):
    X = build_features(df_mau)
    assert list(X.columns) == FEATURE_COLUMNS      # thứ tự cũng phải khớp
    assert X.isna().sum().sum() == 0               # không để lọt giá trị thiếu
```

Ghi nhớ

- Test hợp đồng kiểm tra HÌNH DẠNG và Ý NGHĨA đặc biệt (null, định dạng ngày), không kiểm tra giá trị nghiệp vụ.
- Khi ai đó đổi tên trường, test này vỡ NGAY trên máy — không đợi tới khi người dùng mở trang.
- [DA] Tương đương: test số dòng và tên cột của bảng staging sau bước extract.
- Mỗi test hợp đồng được tính vào số unit test của BT2 (loại “luồng đúng” hoặc “dữ liệu sai”).

PHÂN LOẠI LỖI TÍCH HỢP

Đọc triệu chứng để biết tìm ở đâu

Triệu chứng quan sát được	Nhiều khả năng là	Khoanh vùng nhanh bằng cách
undefined / None / "cannot read property of null"	Lỗi HỢP ĐỒNG: hai bên hiểu khác nhau về tên hoặc hình dạng trường	In ra đối tượng ngay tại ranh giới; so với comment hợp đồng hoặc schema
Chạy không lỗi nhưng số ra sai	Lỗi LOGIC, hoặc lỗi Ý NGHĨA (đơn vị, múi giờ, null bị hiểu là 0)	Tính tay một trường hợp nhỏ; so từng bước trung gian
Trình duyệt báo lỗi CORS, 404 khi gọi API	Lỗi CẤU HÌNH: base URL, cổng, đường dẫn, CORS phía máy chủ	Gọi thử API bằng curl hoặc Postman; kiểm tra .env
Chạy được trên máy mình, máy khác thì không	Lỗi MÔI TRƯỜNG: phiên bản thư viện, biến môi trường, đường dẫn tuyệt đối	Clone repo vào thư mục mới, làm đúng README từng bước
Số dòng sau khi nạp ít hơn số dòng nguồn [DA]	JOIN loại bỏ dòng không khớp khoá	Đếm dòng trước và sau từng bước; tìm khoá không khớp

Lỗi hợp đồng sửa bằng cách THỐNG NHẤT HỢP ĐỒNG (một chỗ), không sửa bằng cách vá thêm if ở bên gọi. Buổi 11 sẽ học kỹ quy trình debug: tái lập → khoanh vùng → sửa → xác nhận.

05

Phần chuyên ngành – Sinh viên tự học theo tài liệu

Frontend gọi API [SE] · kho dữ liệu và dashboard [DA] · huấn luyện và đánh giá [AI]

PHẦN CHUYÊN NGÀNH – SINH VIÊN TỰ HỌC THEO TÀI LIỆU

Việc phải làm ở giai đoạn Sprint 2 theo chuyên ngành

Tài liệu tự học buổi 9 theo track có hướng dẫn chi tiết và ví dụ mã mẫu. Nội dung này áp dụng ngay ở workshop buổi 10.

SE Frontend và tích hợp API

- Dựng frontend bằng React hoặc Vue; tách component theo màn hình đã vẽ ở wireframe
- Gọi API qua một lớp client riêng, không gọi fetch rải rác trong component
- Quản lý trạng thái: đang tải, rỗng, lỗi — ba trạng thái ở phần 2
- Xác thực đăng nhập ở mức cơ bản; lưu token an toàn, không lưu trong mã
- Hiển thị ≥ 3 màn hình chính kết nối API thật, không phải dữ liệu giả

DA Kho dữ liệu và dashboard

- Nạp dữ liệu đã làm sạch từ staging vào bảng đích theo lược đồ đã thiết kế ở BT1
- Bảo đảm nạp lại được: dùng khoá nghiệp vụ để cập nhật thay vì chèn trùng
- Dựng dashboard bằng Streamlit hoặc Power BI với ≥ 2 biểu đồ trả lời câu hỏi phân tích
- Mỗi biểu đồ phải có ngữ cảnh so sánh (kỳ trước, mục tiêu, hoặc trung bình nhóm)
- Xử lý trạng thái rỗng khi bộ lọc không ra dòng nào

AI Huấn luyện và đánh giá mô hình

- Huấn luyện baseline: DummyClassifier theo lớp đa số — BẮT BUỘC có để so sánh
- Huấn luyện ≥ 2 mô hình thật; so sánh trên tập validation, KHÔNG dùng tập test để chọn
- Tính Macro F1, accuracy, F1 từng lớp và confusion matrix
- Đọc confusion matrix: mô hình nhầm giữa cặp lớp nào, và điều đó có ý nghĩa nghiệp vụ gì
- Ghi random seed; lưu mô hình bằng joblib nhưng KHÔNG commit file mô hình

MINI-CASE TẠI LỚP · 12 PHÚT

Tìm sáu vấn đề trong một màn hình “Phiếu quá hạn”

Màn hình nhận được từ một nhóm phát triển

```
PHIEU QUA HAN
id  | customer.name   | due_date            | tech
----+-----------------+---------------------+------
98  | Nguyen Van A    | 2026-07-02 00:00:00 | null
101 | Tran Thi B      | 2026-07-03 00:00:00 | 17

Tổng: 2      CSAT: 3.8612

(Khi tắt máy chủ: màn hình trắng)
(Khi không có phiếu quá hạn: bảng chỉ có dòng tiêu đề)
(Frontend đọc t.customer.name trực tiếp từ bảng customers;
 số điện thoại hiện đầy đủ khi rê chuột vào tên)
```

Nhiệm vụ theo cặp

- **Bước 1 (5 phút):** liệt kê càng nhiều vấn đề càng tốt; gắn mỗi vấn đề với một khái niệm của buổi học (hợp đồng, trạng thái, ngữ cảnh, QT-15...).
- **Bước 2 (4 phút):** chọn ba vấn đề nghiêm trọng nhất và đề xuất cách sửa cụ thể — sửa ở lớp nào, tệp nào.
- **Bước 3 (3 phút):** hai đến ba cặp trình bày; cả lớp bổ sung.
- **Câu hỏi mở rộng:** nếu đây là dashboard [DA] hoặc màn hình gợi ý nhóm lỗi [AI], vấn đề nào vẫn còn nguyên?

TỰ KIỂM TRA NHANH

Năm câu hỏi trước khi rời lớp

1. **Ba thứ mọi hợp đồng dữ liệu phải nêu rõ là gì?**
   Gợi ý: một thứ về hình dạng, một thứ về ý nghĩa, một thứ về những gì xảy ra khi không như mong đợi.

2. **Đổi đơn vị của trường thoiGianXuLy từ giờ sang phút là thay đổi tương thích hay phá vỡ? Vì sao?**
   Gợi ý: tên và kiểu không đổi — vậy điều gì đổi?

3. **Một API trả 500 khi người dùng nhập mô tả lỗi quá ngắn. Sai ở đâu và nên trả gì?**
   Gợi ý: lỗi người dùng hay lỗi hệ thống? Bên gọi cần biết gì để hiển thị?

4. **Kể tên ba trạng thái mọi màn hình và mọi báo cáo phải xử lý; trạng thái nào hay bị quên nhất?**
   Gợi ý: nghĩ tới lúc bộ lọc không ra dòng nào.

5. **Vì sao nên ghép các module tăng dần thay vì ghép một lần rồi mới chạy?**
   Gợi ý: khi luồng vỡ, có bao nhiêu chỗ phải tìm?

KẾT THÚC BUỔI HỌC

Chốt buổi 9 và chuẩn bị cho buổi 10

Tự học, tự nghiên cứu

- Đọc tài liệu tự học buổi 9 theo track và ví dụ mã mẫu.
- Viết hàm chuyển đổi ở ranh giới cho module của mình, kèm comment mô tả hợp đồng dữ liệu.
- Rà soát mã hiện có: có chỗ nào lớp trình bày chạm thẳng vào cấu trúc bảng không? Sửa nếu có.
- Bổ sung xử lý ba trạng thái (đang tải, rỗng, lỗi) cho ít nhất một màn hình hoặc một báo cáo.
- Viết thêm ≥ 1 unit test (ưu tiên test hợp đồng ở ranh giới) — mục tiêu là đủ 3 test thuộc ba loại (luồng đúng · giá trị biên · dữ liệu sai) trước buổi 12.
- Cập nhật README và commit theo quy ước.

Buổi 10 (thực hành) — Workshop module mở rộng & Giao diện (Sprint 2)

- [SE] Frontend ≥ 3 màn hình kết nối API thật; xác thực đăng nhập.
- [DA] Nạp kho dữ liệu; dashboard ≥ 2 biểu đồ có ngữ cảnh so sánh.
- [AI] Huấn luyện baseline và ≥ 2 mô hình; so sánh trên validation; đọc confusion matrix.
- Tích hợp các module: luồng chính chạy end-to-end cục bộ.
- Viết ≥ 5 unit test; cập nhật README.